

Nucleus AI — RBAC Permission Matrix

This document is the canonical reference for the role-based access control (RBAC) system. The machine-readable source of truth lives in `frontend/src/lib/permissions.ts` (`frontend/src/lib/permissions.ts`).

Roles

Role	Rank	Description
SUPER_ADMIN	4	Full, unrestricted access to the entire platform, including system settings and managing other admins.
ADMIN	3	Manages brands, campaigns, documents, and the team (Editors & Viewers). Cannot manage other Admins/Super Admins or system settings.
EDITOR	2	Creates and edits content — brands, campaigns, documents — and runs AI content generation. No delete-brand or user-management rights.
VIEWER	1	Read-only access to brands, campaigns, and documents.

Roles are **hierarchical** by rank for `requireRole` / `isAtLeast` checks, but actual capabilities are governed by the explicit permission lists below.

Permission Matrix

Permission	VIEWER	EDITOR	ADMIN	SUPER_ADMIN
brand:read	✓	✓	✓	✓
brand:create	—	✓	✓	✓
brand:update	—	✓	✓	✓
brand:delete	—	—	✓	✓
campaign:read	✓	✓	✓	✓
campaign:create	—	✓	✓	✓
campaign:update	—	✓	✓	✓
campaign:delete	—	—	✓	✓
document:read	✓	✓	✓	✓
document:create	—	✓	✓	✓
document:update	—	✓	✓	✓
document:delete	—	✓	✓	✓
con- tent:generate	—	✓	✓	✓
user:read	—	—	✓	✓
user:create	—	—	✓	✓
user:update	—	—	✓	✓
user:delete	—	—	✓	✓
user:manage_roles	—	—	✓	✓
admin:access	—	—	✓	✓
audit:read	—	—	✓	✓
settings:manage	—	—	—	✓

User-Management Constraints

Beyond raw permissions, who-can-manage-whom is restricted by

`canManageUserRole(actorRole, targetRole) :`

Actor → Target	VIEWER	EDITOR	ADMIN	SUPER_ADMIN
ADMIN	✓	✓	—	—
SUPER_ADMIN	✓	✓	✓	✓

- **ADMIN** may only create/update/delete **EDITOR** and **VIEWER** accounts and may only assign those two roles.
- **SUPER_ADMIN** may manage every account and assign any role.

`assignableRoles(role) :`

- `SUPER_ADMIN` → `[SUPER_ADMIN, ADMIN, EDITOR, VIEWER]`
- `ADMIN` → `[EDITOR, VIEWER]`
- others → `[]`

Data Scoping

Some read endpoints scope results by ownership:

- **ADMIN / SUPER_ADMIN**: see **all** brands and documents across the platform.
- **EDITOR / VIEWER**: see **only their own** records (`userId === session.user.id`).

Mutating endpoints (`update` / `delete`) additionally verify ownership for

non-admin roles, returning `403` when a non-admin targets a record they do not own.

Usage

Server-side (API routes)

```
import { requirePermission } from '@middleware/rbac';

export async function POST(req: Request) {
  const guard = await requirePermission('brand:create');
  if (!guard.authorized) return guard.response; // 401 or 403
  const { user } = guard; // { id, email, role, status, ... }
  // ... proceed
}
```

Available guards: `requireAuth()` , `requirePermission(p)` ,

`requireAnyPermission([...])` , `requireAllPermissions([...])` , `requireRole(minRole)` .

Client-side (React components)

```
import { usePermissions } from '@/hooks/usePermissions';

function Toolbar() {
  const { can, isAtLeast } = usePermissions();
  return (
    <>
      {can('brand:create') && <CreateBrandButton />}
      {isAtLeast('ADMIN') && <AdminPanelLink />}
    </>
  );
}
```

Default Seed Accounts

Role	Email	Password
SUPER_ADMIN	superadmin@nucleus-ai.com	SuperAdmin123!
ADMIN	admin@nucleus-ai.com	admin123
EDITOR	editor@nucleus-ai.com	editor1234
VIEWER	demo@nucleus-ai.com	demo1234

⚠ Change all default passwords before deploying to production.